Documentação do Projeto: Bilheteria de Cinema (POO)

Projeto de Programação Orientada a Objetos · Prof. Jheymesson A. Cavalcanti Repositório eduardovcmalta/Trabalho-de-POO-2026.2 · branch dudu · commit 79e4d7f · posição em 30/10/2026

1. Resumo executivo

O projeto é um sistema de bilheteria de cinema em Java, entregue em quatro etapas. Em 30/10/2026 as Etapas 1, 2 e 3 estão completas na branch dudu. A Etapa 4 (padrão de projeto e apresentação) ainda não foi iniciada e ficará com a equipe.

Indicador	Resultado
Checklist	22 de 24 itens concluídos (92%); os 2 restantes são da Etapa 4
Código	19 arquivos Java, cerca de 1.500 linhas, nos pacotes Classes, Enums, Interfaces e Persistencia, mais o Main
Compilação e execução	Compila sem erros. O Main roda sem erro e com a mesma saída em execuções repetidas
Verificador de requisitos	45 verificações, 45 passam
Auditoria de modificadores	13 classes: todos os atributos private, todos os métodos public, get/set completos
Ressalvas	12 registradas na seção 9; 6 em aberto, dependentes de decisão ou de confirmação com o professor

As três ressalvas de maior peso:

- O erro de horário passado depende de uma marcação manual (Sessao.setHorarioPassado), porque o slide da Etapa 1 manda não comparar com a hora atual e o da Etapa 3 exige o erro.
- O catálogo de filmes do Repositorio não está ligado às sessões das salas.
- O slide diz que a compra deve ter um ou mais bilhetes, mas o código permite uma Compra só com produtos ou vazia.

2. Conferência com os slides do professor

Os quatro PDFs (Partes 1 a 4) foram relidos e cada exigência foi cruzada com o código da branch dudu.

Parte 1: Classes e encapsulamento

Exigência	Implementação	Status
Classes Usuario, Sala, Sessao, Filme, Bilhete e Compra, com construtores, get/set, atributos private e métodos public	Pacote Classes; auditoria por reflexão	Feito
Usuario com user, CPF, senha, idade, sexo, email, nome, número e código do cartão	Usuario.java	Feito
Usuário realiza, altera e cancela compra	realizarCompra, alterarCompra, cancelarCompra	Feito
Filme com nome, duração, sinopse e valor	Filme.java	Feito
Bilhete com user, CPF, sala, sessão, filme, valor calculado e cadeira	Bilhete.java (guarda também o horário)	Feito
Compra com itens de bomboniere (estoque infinito) e um ou mais bilhetes	Compra.java	Feito (ver ressalva 3)
Compra de bilhetes em método à parte	Compra.comprarBilhete	Feito
Usuário busca as sessões em cada sala	Sala.exibirSessoesDisponiveis	Feito
Um bilhete: poltrona livre. Vários: sugestão de cadeiras juntas	Sessao.sugerirCadeiras, Compra.comprarVariosBilhetes e comprarBilhetes	Feito
Sala com lista fixa de sessões; horário sem sessão é nulo e não aparece	Sala com 7 horários (08h às 22h)	Feito
Sessão com o filme e matriz de cadeiras 10 x 15	Sessao: boolean[10][15]	Feito

Parte 2: Herança e sobrescrita

Exigência	Implementação	Status
Enum de produtos (5 ou 6), com nome (String) e valor (double), integrado à Compra	Produtos	Feito
Enum CupomPromocional com 3 cupons	CupomPromocional	Feito
Enum de salas com tipo e multiplicador	TipoSala	Feito
Procedimento: escolha da sala e depois do filme/sessão	Main	Feito
Estudante herda de Usuario e paga metade	Estudante.calcularValorBilhete	Feito
Critico herda de Usuario, tem origem e bilhete sempre zero	Critico	Feito
Filme com nota (double), quantidade_criticos (int) e vetor critica de 100 posições	Filme.java	Feito
Classe Critica com nome, origem e mensagem	Critica.java	Feito
atribuirNota (0 a 10, nova média) e atribuirCritica (grava no fim do vetor)	Critico	Feito
Sobrecarga dos dois métodos de compra com CupomPromocional	Compra	Feito

Parte 3: Classes abstratas e exceções

Exigência	Implementação	Status
VendasException tratando 3 erros de compra	Classes.VendasException e validações em Sessao e Compra	Feito (ver ressalva 1)
Classe abstrata Base com nome, idade, email e abstratos adicionarUsuario e alterarUsuario	Base.java	Feito
Funcionario concreta, com salário, implementando a interface de filmes	Funcionario.java	Feito
Administrador concreta, com salário e ID, excluir usuário e gerência de filmes	Administrador.java	Feito

Exigência	Implementação	Status
Interface de gerência de filmes (incluir, excluir, alterar)	Interfaces.GerenciaFilmes	Feito
Persistência de usuários e filmes sob responsabilidade do administrador	Persistencia.Repositorio, acionado pelo Administrador	Feito

Parte 4: Padrões de projeto

Exigência	Status
Escolher e aplicar um padrão de projeto, com justificativa	A fazer (equipe)
Apresentação de 8 minutos mais 10 de perguntas; chegar no início da aula nesses dias	A fazer (equipe)

Pontos dos slides que merecem atenção

- A Parte 1 diz que a compra deve ter um ou mais bilhetes. O código não impõe isso.
- A Parte 1 manda não comparar o horário atual com o da sessão, mas a Parte 3 exige o erro de horário que já passou.
- As datas são inconsistentes: as capas mostram 08-2026, 06-2026 e xx-05-2026; os calendários mostram entregas em 10/03, 27/03 e 24/04 de 2026; o repositório tem commits de setembro e outubro de 2026.
- A Parte 1 informa 30% para a Etapa 1 e diz que as entregas 1 e 2 contam para o primeiro GQ. O lembrete da Parte 4 sobre as 4 entregas está com a frase cortada, então o peso de cada etapa não está claro.
- A Parte 1 diz que heranças e sobrecargas entram na primeira entrega, enquanto o calendário da Parte 2 mostra segunda e terceira entregas. A sequência de entregas é ambígua.
- Os slides de Funcionário e Administrador repetem a frase a classe base é uma classe concreta. Foi interpretado como classes concretas que herdam de Base.

3. Arquitetura

Organização dos arquivos

```
Main/src/
  Main.java                  demonstração completa
  Classes/                   Usuario, Estudante, Critico, Critica, Sala,
Sessao, Filme,
                             Bilhete, Compra, Base, Funcionario,
Administrador, VendasException
  Enums/                     Produtos, CupomPromocional, TipoSala
  Interfaces/                GerenciaFilmes
  Persistencia/              Repositorio
```

Relações entre as classes

- Herança: Estudante e Critico estendem Usuario. Funcionario e Administrador estendem Base (abstrata) e implementam GerenciaFilmes.
- Composição: Sala tem 7 Sessao e um TipoSala. Sessao tem um Filme e a matriz de cadeiras. Filme tem até 100 Critica. Compra tem um Usuario, uma lista de Bilhete e uma lista de Produtos com os valores pagos. Bilhete liga Usuario, Sala, Sessao e Filme.
- Persistência: o Repositorio guarda as listas de Usuario e Filme em memória e as grava em arquivos de texto. Funcionario e Administrador recebem o Repositorio no construtor.
- Polimorfismo: calcularValorBilhete é sobrescrito em Estudante (metade) e Critico (zero).

4. Regras de negócio

Preço do bilhete

valor final = valor base do Filme × multiplicador da Sala × fator do usuário − desconto do cupom

O fator é 1 para Usuario, 0,5 para Estudante e 0 para Critico. Com um filme de R$ 20,00, o bilhete custa R$ 20,00 na sala Comum, R$ 24,00 na 3D, R$ 26,00 na XD e R$ 28,00 na XD/3D. Com o cupom BEMVINDO10, o bilhete de R$ 20,00 passa a R$ 18,00 e a pipoca de R$ 10,00 passa a R$ 9,00.

Compra de bilhetes

- Um bilhete: o cliente vê as sessões (Sala.exibirSessoesDisponiveis) e o mapa de cadeiras (Sessao.exibirMapa) e escolhe uma poltrona livre.
- Vários bilhetes: Sessao.sugerirCadeiras devolve o primeiro bloco de cadeiras livres e consecutivas na mesma fileira, da A para a J. Compra.comprarVariosBilhetes compra a sugestão. Compra.comprarBilhetes permite confirmar uma lista de cadeiras, em modo tudo ou nada: se uma estiver ocupada, nenhuma é reservada. Para mais de 15 cadeiras não há como sentar juntos e o sistema lança VendasException.
- Cupons: as duas formas de compra (bilhete e produto) têm sobrecarga com CupomPromocional.

Alterar e cancelar compra

- Usuario.realizarCompra registra a compra atual do usuário.
- Usuario.alterarCompra cancela a compra anterior (liberando as cadeiras) e registra a nova. Se a compra informada já for a atual, o sistema recusa.
- Usuario.cancelarCompra libera as cadeiras e esvazia a Compra, que deixa de listar bilhetes e de ter total.

Os três erros de VendasException

Erro	Quando ocorre	Mensagem
Poltrona ocupada	A cadeira já foi reservada na sessão	ERRO: A poltrona A3 já foi selecionada e está ocupada!
Horário passado	Sessao.horarioPassado está marcado como verdadeiro	ERRO: O horário desta sessão já passou!
Filme fora de cartaz	Filme.emCartaz é falso	ERRO: O filme 'Batman' não está mais em cartaz!

A validação do filme e do horário ocorre antes de reservar qualquer cadeira. Por isso uma compra inválida não deixa cadeira reservada.

Quem faz o quê na gestão

Ação	Funcionario	Administrador
Adicionar e alterar usuário	Sim, em memória	Sim, e grava em arquivo
Incluir, excluir e alterar filme	Sim, em memória	Sim, e grava em arquivo

Ação	Funcionario	Administrador
Excluir usuário	Não	Sim
Salvar e carregar arquivos	Não	Sim

5. Persistência

A persistência usa arquivos de texto, uma das opções listadas no slide da Etapa 3. A classe Persistencia.Repositorio mantém listas dinâmicas e grava na pasta informada no construtor.

Arquivo	Conteúdo de cada linha
usuarios.txt	TIPO;user;cpf;senha;idade;sexo;email;nomeCartao; numCartao;cvv e, para crítico, ;origem. O TIPO é USUARIO, ESTUDANTE ou CRITICO
filmes.txt	nome;duracao;sinopse;valor;emCartaz;nota;qtdCri ticos
criticas.txt	nomeFilme;nomeCritico;origem;mensagem

- A chave de um usuário é o CPF e a de um filme é o nome. Duplicatas são recusadas.
- Ao carregar, o tipo do usuário (Usuario, Estudante ou Critico) e a origem do crítico são restaurados, assim como emCartaz, nota, quantidade de críticos e os comentários do filme.
- Um ponto e vírgula dentro de um texto vira vírgula, para não quebrar o formato. Linhas inválidas são ignoradas com aviso.
- O Main usa a pasta dados_demo, que o .gitignore ignora (assim como dados).
- Não são gravados: compras, bilhetes e cadeiras ocupadas. O enunciado pede persistência só de usuário e filme.

6. Como compilar e executar

Na raiz do projeto, no terminal do VS Code:

```
javac -encoding UTF-8 -d out -sourcepath Main/src Main/src/Main.java
java -cp out Main
```

O Main cria a pasta dados_demo ao rodar. Ele demonstra, em ordem: sessões e mapa de cadeiras; compra de João (bilhete e água); estudante e crítico; erro de poltrona ocupada; vários bilhetes com cupom; alterar e cancelar compra; erros de horário passado e de filme fora de cartaz; e a persistência (incluir, salvar, reiniciar e carregar, excluir e alterar).

Cenário do Main	Resultado esperado
João: bilhete mais água	R$ 25,00
João: com pipoca e cupom BEMVINDO10	R$ 34,00
Estudante (Fátima)	bilhete de R$ 10,00
Crítica (Ana)	bilhete de R$ 0,00
Notas 9,0 e 7,0 no mesmo filme	nota 8.0 com 2 críticos
Maria: 4 bilhetes juntos com cupom	R$ 72,00
Maria altera para 2 bilhetes	R$ 40,00
Persistência	3 usuários (Usuario, Estudante, Critico) e 3 filmes recarregados

7. Histórico do trabalho

Linha do tempo na branch dudu

Data	Commit	O que mudou
17 a 24/09	e61a9c4 a 0345ef5	Esqueleto da Etapa 1, enums, Critico e Critica (Darwin e Guilherme)
27/10	3124a50	.gitignore; remove out/ e .idea/
27/10	34b4a6f e 86510c6	Corrige exibir sessões, cartão como String, nota como double
27/10	96ef1ce	Renomeia Cupons para CupomPromocional
27/10	a6c9caf	Move atribuirNota e atribuirCritica para Critico; origem private
27/10	b141cfe e 2e83397	Sobrecarga de comprarProduto com cupom; testes da Etapa 2

Data	Commit	O que mudou
28/10	971e9a4	Etapa 1: sugestão de cadeiras, métodos de compra no Usuario, getters e setters
28/10	7c73165, 69709a7, 51570d2	Etapa 3: Base, Administrador, Funcionario, interface, persistência em txt e VendasException
28/10	3b880ee	horarioPassado passa a private
30/10	79e4d7f	Correções das Etapas 1 a 3 e persistência via Repositorio

Análises feitas e o que cada uma encontrou

1. Primeira análise: a main estava parada na Etapa 1. A branch guilherme (que continha a darwin) tinha a Etapa 2 parcial. Foram apontados 9 problemas, entre eles exibir sessões imprimindo Filme@hash, cartão como int, atributo origem sem private, métodos de crítica na classe errada e out/ e .idea/ versionados.
2. Segunda análise, após os commits de 27/10: a Etapa 2 estava completa. Os bugs da primeira análise tinham sido corrigidos.
3. Terceira análise, após os commits de 28/10: Etapas 1 e 3 implementadas, com lacunas. O Funcionario só imprimia lógica pendente. A persistência só gravava, duplicava CPF e perdia tipo, origem, emCartaz, nota e críticas. Faltavam getCadeiras e uma forma de ver as cadeiras livres. Cancelar compra deixava a Compra com bilhetes. Faltavam 2 horários na Sala e havia métodos private.
4. Pacote de correções, aplicado pela equipe no commit 79e4d7f: novo Repositorio, Funcionario e Administrador completos, Sessao e Compra ampliadas, Sala com 7 horários, métodos public e Main com demonstração.
5. Verificação final sobre o código do GitHub: branch idêntica ao pacote validado (a única diferença é a quebra de linha no fim do .gitignore); 45 de 45 requisitos; auditoria limpa.

Erros do assistente durante o processo

Por transparência, estes pontos foram corrigidos ao longo do trabalho:

- Foi afirmado que a Etapa 2 estava correta sem reabrir os enums naquela rodada. Depois foi feita uma auditoria item a item, e os enums estavam corretos.
- O primeiro pacote de correções tinha dois métodos private (Sala.obterHorario e Compra.validarSessao), mas a resposta dizia que estavam public. Isso foi percebido na revisão seguinte e corrigido no pacote final, que passou a ser validado depois de extraído, aplicado sobre a dudu limpa.

- Uma das primeiras execuções do verificador mostrou 2 falhas. Eram erro de contagem do teste, e não do código; a causa foi isolada antes de seguir.
- Foi preparada uma prova de conceito da Etapa 4 com Singleton. Ela não foi incluída no pacote final nem na branch, já que a equipe vai fazer a Etapa 4.

8. Verificação e resultados

Verificação	Como foi feita	Resultado
Compilação	Compilador Java sobre todos os arquivos da dudu	Sem erros
Execução do Main	Duas execuções seguidas, comparando as saídas	Sem erro, sem stack trace, saídas idênticas
Auditoria de modificadores	Reflexão em 13 classes	Nenhum atributo fora de private, nenhum método fora de public, nenhum get/set faltando
Verificador de requisitos	45 verificações automáticas (Req.java)	45 passam, 0 falham
Identidade com o pacote	Comparação arquivo a arquivo	Idêntico, salvo a quebra de linha final do .gitignore

O verificador cobre: matriz 10 x 15 e sala com sessões nulas; valor por tipo de sala; compra de um e de vários bilhetes (cadeiras juntas e tudo ou nada); realizar, alterar e cancelar compra; produtos e total; enums; Estudante e Critico; nota, média e críticas; sobrecargas com cupom; os 3 erros de VendasException; Base abstrata, Funcionario, Administrador e interface; persistência com recarga e exclusão.

O verificador (extra/Req.java) veio no pacote de correções e não faz parte do repositório. Ele reflete a leitura dos slides e não substitui o critério do professor.

9. Ressalvas e pontos de atenção

Nada abaixo impede o cumprimento dos slides das Etapas 1 a 3, mas convém decidir ou confirmar com o professor.

#	Ressalva	Recomendação	Situação
1	Horário passado: o erro depende de uma marcação manual (Sessao.setHorarioPassado). Nada compara com a hora atual. A Etapa 1 manda não comparar e a Etapa 3 exige o erro	Confirmar com o professor. Se ele esperar comparação real, ligar a sessão a uma hora de início	Em aberto
2	O catálogo de filmes do Repositorio não está ligado às Sessao e Sala. Excluir um filme no administrador não tira a sessão do ar; isso só acontece com Filme.setEmCartaz(false)	Sem requisito. Estar pronto para explicar na apresentação	Em aberto
3	O slide diz que a compra deve ter um ou mais bilhetes, mas Compra pode existir só com produtos ou vazia	Manter (o resumo mostra nenhum ingresso) ou exigir 1 bilhete antes de produtos	Em aberto
4	Datas inconsistentes nos slides (capas, calendários e commits)	Confirmar os prazos de cada entrega com o professor	Em aberto
5	Peso e avaliação: 30% para a Etapa 1; entregas 1 e 2 para o primeiro GQ; frase da Parte 4 cortada	Confirmar como cada etapa é avaliada	Em aberto
6	Senha, número e código do cartão ficam em texto puro nos arquivos	Aceitável em trabalho acadêmico; citar como limitação conhecida	Aceito
7	Compras, bilhetes e cadeiras ocupadas não são gravados	O enunciado pede só usuário e filme	Aceito
8	Funcionario altera em memória; só o Administrador grava em arquivo	Segue o slide. Mostrar na demonstração: o funcionário inclui e o administrador salva	Aceito
9	A sugestão de cadeiras usa o primeiro bloco livre da mesma fileira (A para J), com no máximo 15 cadeiras	Se quiserem, preferir blocos perto do centro	Aceito
10	A main está 19 commits atrás da dudu, por decisão da equipe. As branches guilherme e darwin estão contidas na dudu e obsoletas	Merge dudu para main só no final; depois apagar guilherme e darwin	Decidido

#	Ressalva	Recomendação	Situação
11	Os métodos auxiliares private do Repositorio (limpar, serializar, lerUsuario, lerFilme) estão fora das classes listadas no enunciado	Se quiserem seguir a regra métodos public à risca, torná-los public	Aceito
12	Risco residual de interpretação dos slides diferente da do professor	Rodar uma demonstração completa antes da entrega	Em aberto

10. Pendências e próximos passos

1. Confirmar com o professor os itens em aberto das ressalvas 1, 3, 4 e 5.
2. Etapa 4: escolher um padrão de projeto, aplicá-lo ao código e escrever a justificativa. Candidatos que combinam com o código atual: Singleton ou Repository para o Repositorio (hoje recebido por construtor por Funcionario e Administrador), Strategy para os descontos de Estudante, Critico e cupom (a herança atual já cobre o requisito, então o ganho é menor) e Factory para criar os tipos de usuário.
3. Preparar a apresentação de 8 minutos mais 10 de perguntas. Sugestão de roteiro: visão geral e arquitetura (2 min), demonstração do Main (3 min), padrão de projeto e justificativa (2 min), limitações e ressalvas (1 min).
4. Merge dudu para main somente com a Etapa 4 pronta. Depois, apagar as branches guilherme e darwin.
5. Manter a planilha de acompanhamento (Status_Projeto_Cinema_POO.xlsx) atualizada: abas Resumo, Checklist, Bugs, Ressalvas, Plano, Branches e Histórico.

11. Apêndices

A. Enums

Enum	Valores
Produtos	PIPOCA R$ 10; NACHOS R$ 20; REFRIGERANTE R$ 8; CACHORRO_QUENTE R$ 8; AGUA R$ 5
CupomPromocional	BEMVINDO10 (10%); QUARTAPROMOCIONAL (30%); PRIORITARIO (20%)
TipoSala	COMUM (Comum, 1.0); TRES_D (3D, 1.2); XD (XD, 1.3); XD_3D (XD/3D, 1.4)

B. Principais métodos públicos

Classe	Métodos
Compra	comprarBilhete (com e sem cupom), comprarBilhetes (com e sem cupom), comprarVariosBilhetes (com e sem cupom), comprarProduto (com e sem cupom), validarSessao, calcularValorTotal, exibirResumoCompra
Sessao	reservarCadeira, cancelarReserva, sugerirCadeiras, converterCadeira, exibirMapa, isCadeiraValida, isCadeiraLivre
Sala	agendarSessao, obterHorario, getHorarioSessao, exibirSessoesDisponiveis
Usuario	calcularValorBilhete, realizarCompra, alterarCompra, cancelarCompra
Critico	calcularValorBilhete, atribuirNota(double, Filme), atribuirCritica(String, Filme)
Base	adicionarUsuario e alterarUsuario (abstratos)
Funcionario	adicionarUsuario, alterarUsuario, incluirFilme, excluirFilme, alterarFilme
Administrador	os mesmos do Funcionario, mais excluirUsuario, salvarDados e carregarDados
Repositorio	adicionarUsuario, removerUsuario, alterarUsuario, buscarUsuario, getUsuarios, adicionarFilme, removerFilme, alterarFilme, buscarFilme, getFilmes, salvar, carregar
GerenciaFilmes	incluirFilme, excluirFilme, alterarFilme